Webhooks

reliable delivery, HMAC signing, retries and duplicates

Watch the 4-min video

In one line: a webhook is the provider calling *your* URL when an event happens; the sender queues events from an outbox, signs them with HMAC and retries for days, and the receiver verifies, records the event ID, answers 200 fast and processes idempotently without trusting order.

1 What a webhook is

A customer pays on your site. Your payment provider needs to tell your backend right away, without you polling every few seconds. So it calls *you*: an **HTTP POST to a URL you registered**. That's a webhook. The idea is simple; making delivery reliable is a classic system design question. It has two sides: the **sender** and the **receiver**.

2 The sender: outbox, queue, workers

When an event happens, like *payment succeeded*, **don't call the customer's URL inside the request** that created the payment. Write the event to an **outbox table in the same transaction** as the payment, then let a queue and a pool of **delivery workers** send it. The transaction guarantees the event is recorded if and only if the payment is. And a slow or broken customer endpoint never slows down your core API.

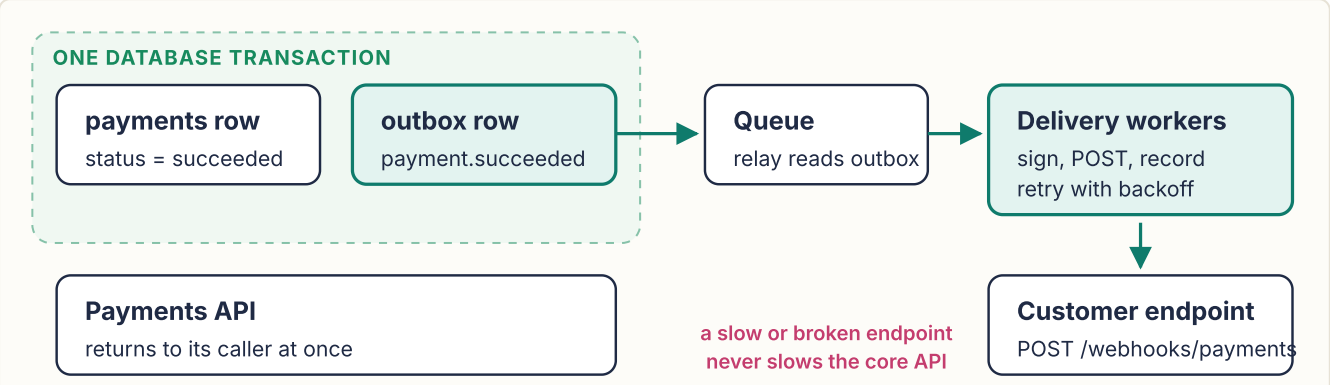


Figure 1. The payment and its event commit together. Delivery happens later, from workers, so the API's latency doesn't depend on anyone's endpoint.

Sign every delivery

The sender computes an **HMAC-SHA256** over a **timestamp and the raw body**, using a **secret unique to each endpoint**, and sends it in a signature header. Receivers verify it, so nobody can forge events. Including the timestamp in the signed string lets receivers reject old, replayed requests.

Put an **event ID**, an **event type**, a **creation time** and an **API version** in the payload, e.g. `{"id": "evt_9", "type": "payment.succeeded", "created": 1730700000, "api_version": "2026-01-01", "data": {…}}`.

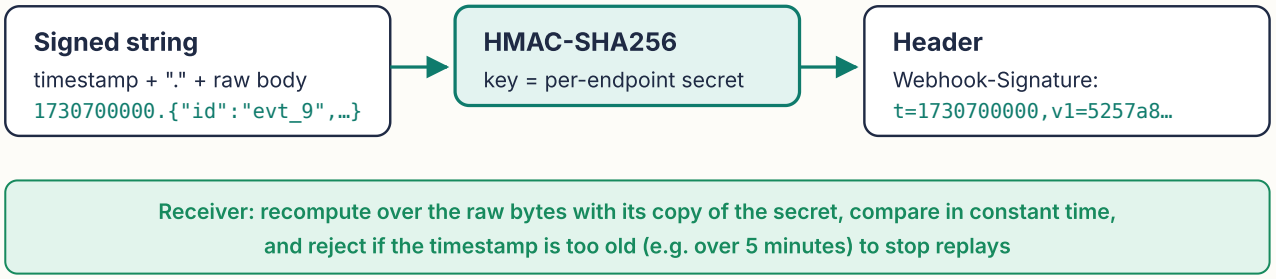


Figure 2. Signing and verifying. The receiver must hash the exact bytes it received; re-serialized JSON changes spacing and breaks the signature.

3 Retries, and visibility for customers

Endpoints fail, so retry. A delivery succeeds only on a **2xx response within a short timeout**, say 10 seconds. Anything else is retried with **exponential backoff and jitter**: for example after 1 minute, 5 minutes, 30 minutes, then hourly, for **up to three days**. After repeated failures, mark the endpoint as failing, **alert the owner**, and eventually **disable** it. Stripe, for example, retries for up to three days.

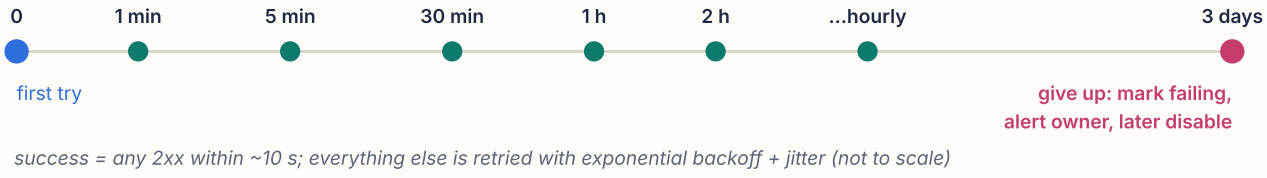


Figure 3. A typical schedule. Long gaps give a customer time to fix an outage; three days of attempts means a weekend outage is still recoverable.

- **Delivery log**: every attempt, its status code, latency and a response snippet.
- **Replay** any event from a dashboard or an API.
- **List endpoint** (e.g. `GET /events?since=…`) so customers can fetch events they missed after a long outage.
- **Verify a URL when it is registered**, with a test event or a challenge, and **block internal addresses** (private ranges such as 10.0.0.0/8, and cloud metadata at 169.254.169.254) to prevent **server-side request forgery** (SSRF).

4 The receiver: verify, store, ack, process

For example, a FastAPI endpoint. **Verify the signature on the raw body.** Then **store the event and return 200 quickly**, within a second or two. Do the real work **asynchronously**, in a background worker. If you do slow work before responding, the sender times out and retries, and you get even more duplicates.

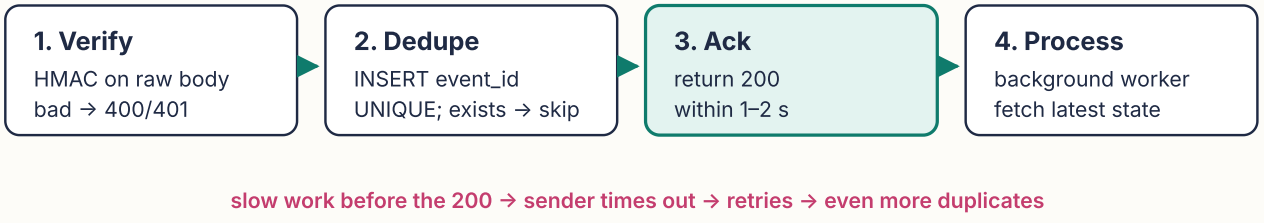


Figure 4. The receiver's job before answering is small and fast. Everything slow happens after the 200.

```
@app.post("/webhooks/payments")
async def hook(request: Request):
    raw = await request.body()                        # bytes, not parsed JSON
    t, sig = parse_header(request.headers["Webhook-Signature"])
    expected = hmac.new(SECRET, f"{t}.".encode() + raw, sha256).hexdigest()
    if not hmac.compare_digest(expected, sig) or time.time() - int(t) > 300:
        raise HTTPException(400)
    event = json.loads(raw)
    inserted = await db.insert_ignore("webhook_events", id=event["id"], body=raw)
    if inserted:
        await queue.enqueue("process_event", event["id"])
    return {"ok": True}                               # 200 either way
```

5 Duplicates and ordering

Delivery is **at least once**, so duplicates will happen: a retry after a timeout, a replay, a sender bug. Make processing **idempotent**: keep a table of processed event IDs with a **unique constraint**, and skip an event you've already handled. Return 200 for a duplicate too, or the sender will keep retrying it.

And **don't assume ordering**. A *payment refunded* event can arrive before *payment succeeded*. Use the event's timestamp or the object's version, or simply **fetch the latest state from the provider's API** before acting.

WORKED EXAMPLE: A REFUND THAT ARRIVES FIRST

At 10:00:00 the provider emits `payment.succeeded` (evt_1); at 10:00:03, `payment.refunded` (evt_2). Your endpoint was down for evt_1's first attempt, so evt_2 arrives first and evt_1 arrives on its 1-minute retry. If you blindly apply events in arrival order, the order ends up *paid*, wrong. Instead, the worker for each event calls `GET /payments/pay_7`, sees `status: refunded`, and writes that. Then evt_1 arrives a minute later, the worker fetches again and still sees refunded. Correct either way.

COMMON MISTAKES

- Calling the customer's URL synchronously inside the API request.
- Verifying the signature over re-serialized JSON instead of the raw bytes, or comparing with `==` instead of a constant-time compare.
- No timestamp check, so captured requests can be replayed.
- Doing slow work before returning 200.
- No event-ID dedupe; assuming events arrive in order.
- Letting customers register internal URLs (SSRF).

6 Key terms

Webhook: server-to-server HTTP POST on an event

HMAC-SHA256: keyed hash proving sender and integrity

Idempotent receiver: same event twice has one effect

Outbox: event row written in the same transaction

At-least-once: every event arrives, maybe more than once

SSRF: tricking a server into calling internal addresses

7 Interview questions

Q1. How do you design a reliable webhook delivery system? BASIC
Write events to an outbox in the same transaction, deliver asynchronously with a queue and workers, sign every payload with HMAC, retry failures with exponential backoff and jitter for days, and keep delivery logs with replay, endpoint verification and auto-disable for dead endpoints.

Q2. How should a receiver handle duplicates? BASIC
Verify the signature, record the event ID with a unique constraint, skip events already processed (still returning 200), respond fast, process asynchronously, and never rely on ordering.

Q3. Why sign webhooks with HMAC, and why include a timestamp? BASIC
Anyone can POST to a public URL. The HMAC proves the request came from someone holding the endpoint's secret and wasn't altered. Signing the timestamp lets the receiver reject requests older than ~5 minutes, so a captured request can't be replayed later.

Q4. Why not call the customer's URL inside the payment request? SCENARIO
Their endpoint might take 10 s or be down; your API would slow down or fail with it, and a crash between commit and call would lose the event. The outbox records the event atomically with the payment and workers deliver it independently.

Q5. A customer's endpoint is down for 6 hours. What happens? SCENARIO
Deliveries fail and are retried on a backoff schedule (1 min, 5 min, 30 min, then hourly); within the 3-day window they succeed once it's back. The customer can also replay from the dashboard or fetch missed events from a list endpoint.

Q6. The receiver's handler takes 15 s to process an order. What goes wrong? SCENARIO
The sender's ~10 s timeout fires, it counts the delivery as failed and retries, so the work runs again and again. Store the event, return 200 in 1–2 s, and process in a background worker.

Q7. Your signature check fails for valid events. Likely cause? TRICKY
Hashing parsed and re-serialized JSON instead of the raw request bytes (key order and spacing change), a framework that consumed the body, or the wrong per-endpoint secret. Hash the exact raw body.

Q8. payment.refunded arrives before payment.succeeded. How do you stay correct? TRICKY
Don't apply events by arrival order. Compare the event's timestamp or object version with what you stored, ignoring older ones, or fetch the latest state from the provider's API before acting.

Next: SD-C-021 · Async APIs for Long Jobs backendroadmap.com